Recommender Systems: Content-Based and Collaborative Filtering

Instructions:

- Complete the coding exercises in a Jupyter notebook.
- Submit your work as a `.ipynb` file.
- Ensure all code is properly commented and executed before submission.
- Plagiarism detected will result in zero marks
- Max marks: 120

Dataset

For this assignment, use the MovieLens Dataset.

Download the dataset here: MovieLens Dataset Download

This dataset contains various versions of movie ratings and metadata, making it a standard benchmark for recommender system research. It includes user ratings, movie information, and user demographic data, which will be useful for implementing and evaluating various recommendation algorithms.

Ensure that all implementations and evaluations are based on this dataset for consistency across all submissions.

Part 1: Content-Based Filtering

20 Marks

Task 1: Implementing TF-IDF Based Recommendation

In this task, you will build a content-based recommender system using Term Frequency-Inverse Document Frequency (TF-IDF). The goal is to recommend movies similar to a given movie based on textual features.

1. Load the dataset and extract the movie descriptions (list of genres).
2. Compute TF-IDF vectors using `TfidfVectorizer` from `sklearn`.
3. Compute cosine similarity between items (may use `sklearn.metrics`)
4. Implement a Recommendation Function
   - Create a function that takes a movie title and returns the top-N most similar movies.
5. Test your recommender with sample queries
   - Input a few movie titles and print the top-5 recommended movies along with their cosine similarity scores.

Task 2: User-Profile-Based Content Recommender

In this task, you will build a content-based recommender system that personalizes recommendations based on user preferences. Instead of relying solely on item descriptions, you will construct *user profiles* from their historical interactions (i.e., rated movies) and use them to suggest relevant movies.

Constructing User Profiles

Each user has rated multiple movies, providing a signal of their preferences. For this exercise, user profile is a weighted combination of the movies they have rated. The idea is:

- If a user rates a movie highly, that movie's features should contribute more to their profile.
- If a user rates a movie poorly, it should contribute less (or not at all).

Mathematical Formulation
Let:

- u be a user.
- M be the set of movies the user has rated.
- $r_{u,m}$ be the rating given by user u to movie m.
- f_m be the feature vector for movie m.

The user profile vector P_u is computed as:

$$P_u = \frac{\Sigma_{m \in M}(r_{u,m} \cdot f_m)}{\Sigma_{m \in M}(r_{u,m})}$$

A simple way to compute this is by taking a weighted average of the movie's TF-IDF vectors calculated previously, where the weights are the user's ratings. Therefore, you will:

1. Filter the genres (TF-IDF vectors) of the movies a user has rated.
2. Compute a weighted genre preference for each user by multiplying each movie's feature vector by the user's rating and adding the weighted vectors.
3. Normalize the preference scores to ensure fair comparisons across users. (Normalize by total rating sum for each user)

Computing User-Movie Similarity

Once user profiles and movie vectors are created, recommendations can be generated by:

- Computing cosine similarity between the user profile and all movie vectors.

$$Similarity(P_u, f_m) = \frac{P_u \cdot f_m}{||P_u|| \cdot ||f_m||}$$

- Ranking movies based on similarity scores.
- Returning the top-N most relevant movies for each user.

Evaluating Personalization

To measure the quality of user-profile-based recommendations:

- Compare recommended movies with movies a user actually rated highly.
- Compute precision@K and recall@K using their watched movies as ground truth.

Part 2: Collaborative Filtering

20 marks

Task 3: User-Based Collaborative Filtering

User-based collaborative filtering recommends items based on the preferences of similar users. The assumption is that users with similar tastes in the past will have similar preferences in the future.

1. Populate a user-movie rating matrix from the dataset
2. Compute user-user similarity
   - Calculate pairwise similarity between users using either: cosine similarity or Pearson correlation
   - ~ **Hint:** Consider which is a more suitable metric if user ratings have different biases (e.g., some users rate generously while others rate harshly).
3. Predict ratings for a given user using weighted average of neighbors
   - Select the K most similar users.
   - Compute the weighted average of their ratings, giving more weight to users with higher similarity scores.
   - Predict ratings for movies the user has not rated.
4. Implement a recommendation function based on predicted ratings
   - Rank movies based on the predicted ratings.
   - Recommend the top-N highest-rated movies for the target user.
5. Evaluate the system with RMSE (Root Mean Squared Error), Precision@K, and Recall@K
6. Test with different values of K (number of similar users) to see how recommendation quality changes.

Task 4: Item-Based Collaborative Filtering

Item-Based Collaborative Filtering recommends items based on their similarity to items a user has previously liked or rated highly. The key idea is: If two items are rated similarly by many users, they are likely to be similar.

1. Compute item-item similarity using Pearson correlation or cosine similarity
2. Predict ratings for a given user based on similar items
   - Identify items the user has already rated.
   - Find similar items and use their ratings to predict new ratings.
   - Compute the weighted average of similar item ratings, where the weights are similarity scores.
   - ~ **Hint:** Higher similarity = higher weight in rating prediction.
3. Generate top-N movie recommendations for a user
4. Evaluate the system and compare the recommendations with user-based CF to analyze differences in output.
5. Do you think item based CF may be faster or more memory-efficient than user based CF in the real world? Why?

Part 3: Matrix Factorization for Recommender Systems

20 marks

Task 5: Implementing SVD for Recommendations

In this task, you will perform Singular Value Decomposition (SVD) on the user-item rating matrix. Use the factorized matrices to predict missing ratings. Generate recommendations based on reconstructed ratings. And compare the performance of SVD vs. user-based and item-based CF.

1. Prepare the user-item matrix for factorization
   - Populate the user-item matrix, filling missing ratings with a placeholder (e.g., mean rating per user or zero).
   - Normalize the data (optional but improves performance).
2. Apply Singular Value Decomposition (SVD) using `scipy` or `numpy` functions
   - Factorize the user-item matrix **R** into three matrices:

$$R \approx U \cdot \Sigma \cdot V^T where:$$

U = User latent factor matrix (users represented as feature vectors).
Σ = Diagonal matrix of singular values (importance of latent factors).
V^T = Item latent factor matrix (items represented as feature vectors).

3. Predict missing ratings
   - Reconstruct the approximate rating matrix:

$$\hat{R} = U \cdot \Sigma_k \cdot V^T$$

   - Use the reconstructed ratings to suggest top-N recommended movies for a given user
   - ~ **Hint:** Ensure that predictions are within the rating scale (e.g., 1-5). Clip values if necessary.
4. Evaluate SVD-based recommendations and compare with item-based and user-based CF. Use RMSE, Precision@K, and Recall@K

Task 6: Implementing Matrix Factorization with the Surprise Library

The `Surprise` library is a popular Python package for recommendation algorithms, including SVD, KNN, and other matrix factorization techniques.

1. Load the dataset into the surprise framework. Split it into train-test sets
2. Train an SVD model using surprise and tune the hyperparameters
3. Evaluate model performance with RMSE, Precision@K, and Recall@K
4. Compare Surprise's SVD implementation vs. the SVD implementation from task 5.

Part 4: Hybrid Recommendation Model

10 marks

Task 7: Implementing a Hybrid Recommendation Model

In this task, you will implement a hybrid recommender system that combines content-based filtering and collaborative filtering and compare them with individual recommendation methods. Multiple hybridization strategies exist, for example a weighted average may be taken from the CF scores and CBF scores as shown below.

$$\text{Final Score} = \alpha \times \text{CBF Score} + (1 - \alpha) \times \text{CF Score}$$

where α is a tunable weight (e.g., 0.5 for equal contribution).

1. For this task, you will implement a meta-learning strategy. I.e. train a machine learning model to decide how to blend CF and CBF scores dynamically.
   - For each user-movie pair, our model may be provided with
     - Predicted score from CBF
     - Predicted score from CF
     - Movie popularity score (average rating of the movie)
     - User's average rating (to account for user bias)
   - Use historical user ratings as ground truth and train a meta-model to predict how a user will rate a movie
2. Evaluate the hybrid model using RMSE, Precision@K, and Recall@K. Compare against CF and CBF recommender systems
3. Analyze performance on cold-start users.

Part 5: Learning-Based Recommender Systems

40 marks

Task 8: Content-Based Filtering with a Neural Network

In this task, you will implement content-based filtering using a neural network by:

- Using movie metadata (genres, release year, and engineered features like average ratings) as inputs.
- Generating user and movie feature vectors using a neural network.
- Training the model on user ratings to predict preferences.

Steps:

1. Load and process the movie lens dataset
   - Extract movie metadata (specified above) and user metadata (average rating per genre)
2. Create neural networks for generating user and item embeddings.
   Instead of manually constructing latent factors (as in matrix factorization), the neural networks will learn embeddings from metadata.
   - **Movie Embeddings:**
     - Input: Movie features (one-hot encoded genres, release year, average rating).
     - Processing: A dense neural network converts these features into a low-dimensional vector representation (embedding).
   - **User Embeddings:**
     - Input: User features (average rating per genre).
     - Processing: A dense neural network generates a user vector capturing preferences.
3. Prepare the complete neural network
   - Each training sample consists of:
     - User vector (metadata-based)
     - Movie vector (metadata-based)
     - Target output: User's actual rating for the movie
   - The neural network will have:
     - Two input branches (user features and movie features).
     - Dense layers that process each input separately.
     - Merging layer that combines user and item embeddings.
     - Final layer that predicts the rating.

> **Neural Network Flow:**
>
> User Feature Input → Dense Layer → User Embedding
> Movie Feature Input → Dense Layer → Movie Embedding
> Combine User & Movie Embeddings → Predict Rating

4. Train the model on user ratings
   - Use MSE (Mean Squared Error) loss
   - Train using Adam optimizer and tune the learning rate.
   - **Hint:** Monitor **validation loss** to avoid overfitting.
5. Evaluate performance and compare with traditional content-based filtering (cosine similarity on genres). Does the neural model capture more complex user preferences than standard TF-IDF-based filtering?

Task 9: Reinforcement Learning in Recommender Systems

In this task, you will:

- Implement a simple reinforcement learning-based recommender.
- Explore how RL optimizes long-term engagement instead of just immediate user satisfaction.
- Compare RL-based recommendations with traditional models.

Understanding RL for Recommendations

A reinforcement learning-based recommender treats the recommendation process as a sequential decision-making problem:

- **Agent:** The recommendation system.
- **State:** User's past interactions (e.g., previously watched/rated movies, genre preferences).
- **Action:** A set of movies recommended to the user.
- **Reward:** User feedback (e.g., watch time, click, or high rating).
- **Policy:** A strategy to balance exploration (new items) and exploitation (similar items).

Steps:

1. Define the environment (user-item interaction space)
2. Implement a Multi-Armed Bandit (MAB) model
   - Maintain a table of movie reward estimates.
   - The system chooses movies (arms) based on estimated rewards.
   - Uses an exploration-exploitation strategy:
     - **ε-Greedy:** Recommend top movies most of the time, but randomly explore some new movies.
     - **UCB (Upper Confidence Bound):** Prioritizes less explored movies that may have high rewards.
     - ~ **Hint:** Start with ε-Greedy, where ε = 0.1 means 10% of recommendations explore new movies.
   - Simulate feedback based on movie lens ratings (you may treat rating ≥ 4 as positive reward, rating < 4 as negative reward, and no rating as neutral)
   - Update the movie's reward score based on user feedback.
3. Implement a Q-Learning agent
   - Initialize the Q-Table with random values.
   - Choose an action (recommend a movie) using an ε-Greedy strategy.
   - Observe the reward and update the Q-Table using the Q-learning update rule:

     $$Q(s,a) = Q(s,a) + \alpha\Big[r + \gamma \max Q(s',a') - Q(s,a)\Big]$$

     where:
     α = Learning rate (how quickly Q-values update).
     γ = Discount factor (importance of future rewards).
     r = Immediate reward.
   - Repeat for multiple users and movies until convergence.
4. Compare RL with traditional models
   - Measure how often the model explores new movies vs. recommends known favorites.
   - Compare RL-based recommendations with SVD, user-based CF, and item-based CF.

Part 6: Explainability in Recommender Systems

10 marks

Task 10: Feature-Based Explanations (For Content-Based Filtering)

- Show which features contributed the most to a recommendation.
- Example: *"This movie was recommended because you liked Sci-Fi movies from the 2010s."*
- ~ **Hint** Use SHAP (SHapley Additive Explanations) to show how each feature (e.g., genre, year, rating) influenced a recommendation.

Task 11: Neighborhood-Based Explanations (For Collaborative Filtering)

- Show which similar users or items influenced a recommendation.
- Example: *"Users who liked Inception also liked Interstellar."*
- ~ **Hint:** Use k-Nearest Neighbors (k-NN) explanations based on user similarity.

Task 12: Model-Agnostic Explainability (For Deep Learning Models)

- Use tools like LIME (Local Interpretable Model-agnostic Explanations) to break down neural network decisions.

Task 13: Evaluating Explainability

Compare how different explainability methods affect user understanding.

- Do explanations make recommendations clearer?
- Do explanations reveal biases in recommendations?

Submission Guidelines:

- Ensure all code is well-commented and modular.
- Submit the assignment as a Jupyter notebook.
- Include a README file with instructions on how to run your notebook and any dependencies required.
- Include visualizations and explanations where necessary